

## Paper – Reproducibility code

### i. Figures 3

Generate: cms\_compare\_rho\_0p0.json (In folders K10dg03, K10dg07, K5dg03, K5dg07)

**Then run the following**

```
import importlib

tg_mod = importlib.import_module("add_twogroup_dgm_rho0")
importlib.reload(tg_mod)

tg_mod.N_SIM_OVERRIDE = 200000
tg_mod.CHUNK_SIZE = 2000
tg_mod.SEED_OFFSET = 2000
tg_mod.N_JOBS_OVERRIDE = -1
tg_mod.FORCE_RECOMPUTE = True

tg_mod.main()
```

**It generates:**

```
K10dg03/cms_compare_rho_0p0_twogroup_dgm_fwer.png
K10dg03/cms_compare_rho_0p0_twogroup_dgm_power.png

K5dg03/cms_compare_rho_0p0_twogroup_dgm_fwer.png
K5dg03/cms_compare_rho_0p0_twogroup_dgm_power.png

K10dg07/cms_compare_rho_0p0_twogroup_dgm_fwer.png
K10dg07/cms_compare_rho_0p0_twogroup_dgm_power.png

K5dg07/cms_compare_rho_0p0_twogroup_dgm_fwer.png
K5dg07/cms_compare_rho_0p0_twogroup_dgm_power.png
```

### li Figure S3

**Generate Improved Hommel Threshold**

```
import importlib
import os
import sys

project_dir = "/Users/rajeshkarmakar/Downloads/BUStrongControl"

if project_dir not in sys.path:
    sys.path.insert(0, project_dir)

threshold_mod = importlib.import_module(
    "compute_improved_hommel_thresholds"
```

```

)
importlib.reload(threshold_mod)

threshold_mod.SOURCE_THRESHOLD_FILE = os.path.join(
    project_dir,
    "cms_thresholds_K10.json",
)

threshold_mod.OUTPUT_FILE = os.path.join(
    project_dir,
    "improved_hommel_thresholds_K10.json",
)

# None uses B_threshold=1,000,000 from cms_thresholds_K10.json.
threshold_mod.B_OVERRIDE = None

threshold_mod.CHUNK_SIZE = 20_000

# None uses seed=123 from cms_thresholds_K10.json.
threshold_mod.SEED_OVERRIDE = None

threshold_mod.QUANTILE_METHOD = "higher"

# Do not modify cms_thresholds_K10.json.
threshold_mod.MERGE_INTO_SOURCE = False

threshold_mod.main()

```

**It generates:** improved\_hommel\_thresholds\_K10.json

#### **Run Simulation:**

```

import importlib
import sys

project_dir = "/Users/rajeshkarmakar/Downloads/BUStrongControl"

if project_dir not in sys.path:
    sys.path.insert(0, project_dir)

plot_mod = importlib.import_module(
    "plot_improved_hommel_power_gain"
)
importlib.reload(plot_mod)

plot_mod.main()

```

**It generates:** improved\_hommel\_power\_gain\_K10\_tp\_0p3.png

## 2. Computational Complexity

### i. Figure S7

```
import importlib

fixed_mod = importlib.import_module("run_simulation_from_thresholds")
importlib.reload(fixed_mod)

fixed_mod.rho = 0.0
fixed_mod.n_sim = 10000
fixed_mod.seed = 123
fixed_mod.n_jobs = -1
fixed_mod.main()
```

#### **This Creates:**

```
cms_compare_rho_0p0.json
cms_compare_rho_0p0_fwer.png
cms_compare_rho_0p0_power.png
cms_compare_rho_0p0_time.png
cms_compare_rho_0p0_combined.png
```

### ii. Figure S8

```
runtime_mod = importlib.import_module("benchmark_runtime_by_dimension")
importlib.reload(runtime_mod)

runtime_mod.K_MIN = 2
runtime_mod.K_MAX = 10
runtime_mod.N_SIM = 10000
runtime_mod.RHO = 0.0
runtime_mod.SEED = 123
runtime_mod.N_JOBS = -1
runtime_mod.DATA_TARGET_POWER = 0.7
runtime_mod.ALTERNATIVE_PROBABILITY = 0.5

runtime_mod.main()
```

#### **Files produced:**

```
runtime_by_dimension_pimix_0p7.png
runtime_by_dimension_pimix_0p7_fwer.png
runtime_by_dimension_pimix_0p7_power_tpr.png
runtime_by_dimension_pimix_0p7_combined.png
runtime_by_dimension_pimix_0p7.json
```

## 3. Dependence

## i) Independent BU under dependence ( Figure S4, S5 & S6)

Generating Thresholds:

```
import importlib
import json
import shutil

threshold_mod = importlib.import_module("compute_thresholds_once")
importlib.reload(threshold_mod)

threshold_mod.K = 10
threshold_mod.alpha = 0.05
threshold_mod.procedure_target_powers = {
    "bayes": [0.3, 0.7, 0.95],
    "pil": [0.3, 0.7, 0.95],
}
threshold_mod.alternative = "normal"
threshold_mod.B_threshold = 1_000_000
threshold_mod.seed = 123
threshold_mod.n_jobs = -1
threshold_mod.chunk_size = 2000
threshold_mod.null_correlation_phi = 0.0

# Generates cms_thresholds.json.
threshold_mod.main()

source_file = (
    "/Users/rajeshkarmakar/Downloads/BUStrongControl/"
    "cms_thresholds.json"
)
destination_file = (
    "/Users/rajeshkarmakar/Downloads/BUStrongControl/"
    "cms_thresholds_K10.json"
)

with open(source_file, "r", encoding="utf-8") as f:
    payload = json.load(f)

if payload["params"]["K"] != 10:
    raise ValueError("The generated threshold file is not for K=10.")

shutil.copy2(source_file, destination_file)

print("Created:", destination_file)

It generates: cms_thresholds_K10.json
```

Run simulation to generate files:

```
import importlib
from statistics import NormalDist

run_mod = importlib.import_module("run_simulation_from_thresholds")
```

```

importlib.reload(run_mod)

K = 10
alpha = 0.05
data_target_power = 0.3 ## change it to 0.7 or 0.95

std = NormalDist()

run_mod.THRESHOLD_FILE = (
    "/Users/rajeshkarmakar/Downloads/BUStrongControl/"
    "cms_thresholds_K10.json"
)

run_mod.mu_alt = (
    std.inv_cdf(alpha / K)
    - std.inv_cdf(data_target_power)
)

run_mod.rho = -0.1 ## change it to different values: -0.1, 0.0, 0.3 or 0.7
run_mod.n_sim = 200000
run_mod.seed = 123
run_mod.n_jobs = -1

run_mod.main()

```

**It generates:** cms\_compare\_rho\_-0p1.json

Generates plots from "cms\_compare\_rho\_-0p1.json" file

```

import importlib
import json

sim_mod = importlib.import_module("simulate_cms_normal")
importlib.reload(sim_mod)

json_file = (
    "/Users/rajeshkarmakar/Downloads/BUStrongControl/"
    "K10dg03/cms_compare_rho_-0p1.json"
)

plot_prefix = (
    "/Users/rajeshkarmakar/Downloads/BUStrongControl/"
    "K10dg03/cms_compare_rho_-0p1_custom"
)

with open(json_file, "r", encoding="utf-8") as f:
    payload = json.load(f)

plot_paths = sim_mod.plot_simulation_results(
    payload["results"],
    plot_prefix,
)

for path in plot_paths:
    print("Saved:", path)

```

**It generates:** K10dg03/cms\_compare\_rho\_-0p1\_custom\_fwer.png  
K10dg03/cms\_compare\_rho\_-0p1\_custom\_power.png

## ii) Adapting to Dependence: Simulation (Table S1)

Calibration, B = 200000  
Simulation, N\_sim = 100000  
Simply run:  
python3 run\_configuration\_table.py

## 4. Figure S3

### i. Generate K=10 Improve Hommel Threshold

```
import importlib

threshold_mod = importlib.import_module(
    "compute_improved_hommel_thresholds"
)
importlib.reload(threshold_mod)

threshold_mod.SOURCE_THRESHOLD_FILE = (
    f"{project_dir}/cms_thresholds_K10.json"
)

threshold_mod.OUTPUT_FILE = (
    f"{project_dir}/improved_hommel_thresholds_K10.json"
)

threshold_mod.B_OVERRIDE = 1_000_000
threshold_mod.SEED_OVERRIDE = 123
threshold_mod.CHUNK_SIZE = 20_000
threshold_mod.QUANTILE_METHOD = "higher"

# Do not modify cms_thresholds_K10.json.
threshold_mod.MERGE_INTO_SOURCE = False

threshold_mod.main()
```

**It generates:** improved\_hommel\_thresholds\_K10.json

### ii. Generate Figure S3

```
import importlib

plot_mod = importlib.import_module(
    "plot_improved_hommel_power_gain"
)
importlib.reload(plot_mod)

plot_mod.K = 10
```

```
plot_mod.ALPHA = 0.05
plot_mod.DATA_TARGET_POWER = 0.3
plot_mod.ALTERNATIVE_PROBABILITY = 0.5
plot_mod.N_SIM = 200_000
plot_mod.CHUNK_SIZE = 20_000
plot_mod.SEED = 123

plot_mod.THRESHOLD_FILE = (
    f"{project_dir}/improved_hommel_thresholds_K10.json"
)

plot_mod.OUTPUT_PREFIX = (
    f"{project_dir}/improved_hommel_power_gain_K10_tp_0p3"
)

plot_mod.main()
```

**It generates:** improved\_hommel\_power\_gain\_K10\_tp\_0p3.png

## **5. Generate Table 1 and Table 2**

**Run:** cochrane\_example.ipynb